(19)中华人民共和国国家知识产权局

(12)发明专利

(10)授权公告号 CN 104503966 B
(45)授权公告日 2017.12.12

(21)申请号 201410550641.4

(22)申请日 2014.10.16

(65)同一申请的已公布的文献号
申请公布号 CN 104503966 A

(43)申请公布日 2015.04.08

(73)专利权人 杭州斯凯网络科技有限公司
地址 310013 浙江省杭州市紫荆花路2号联合大厦B座10楼

(72)发明人 周正中

(74)专利代理机构 杭州杭诚专利事务所有限公司 33109
代理人 尉伟敏

(51)Int.Cl.
G06F 17/30(2006.01)

(56)对比文件
CN 101957748 A,2011.01.26,
GB 2505210 A,2014.02.26,
JP 2003271494 A,2003.09.26,
CN 1645336 A,2005.07.27,
张爱国等.基于PostgreSQL数据库的GML数据存储.《测绘科学》.2008,

审查员 汪安

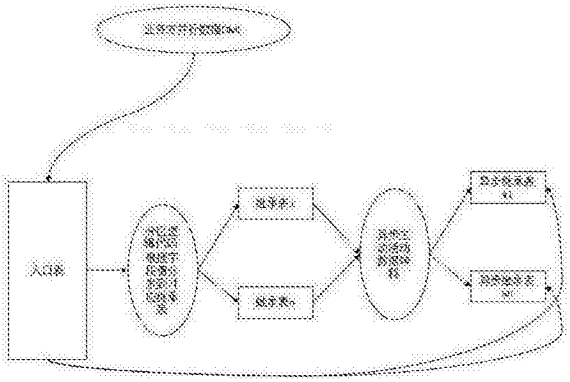
权利要求书1页 说明书5页 附图2页

(54)发明名称

PostgreSQL大数据高效免维护自动分区方法

(57)摘要

本发明公开了一种PostgreSQL大数据高效免维护自动分区方法，属于数据库大数据处理技术领域，创建入口表，将分区值抽象转换为一个随时间推移的固定循环值；根据固定的循环值创建固定的分区逻辑；创建根据入口表的分区逻辑代码和字段值分发的分区继承表；把活动分区从分区逻辑中剥离，让活动分区随着时间的推移进行自动创建；利用继承特性将固定分区的数据定时的转移到活动分区，并在转移完成后删除固定分区中的数据；根据数据量进行分裂设计，预先定义好分层和数据量对应规则，当活动分区表数据量超过上限阈值则自动往下一层分裂；根据分区继承表的异步主动活动的数据转移逻辑来创建异步继承表。



CN 104503966 B

1.一种PostgreSQL大数据高效免维护自动分区方法,其特征在于,包括如下步骤:

(1)创建入口表,且应用程序只与入口表交互;

(2)按照数据库分区表的分区值特征,将分区值抽象转换为一个随时间推移的固定循环值;

(3)根据固定的循环值创建固定的分区逻辑;

(4)创建根据入口表的分区逻辑代码和字段值分发的分区继承表;

(5)把活动分区从分区逻辑中剥离,让活动分区随着时间的推移进行自动创建;

(6)利用继承特性将固定分区的数据定时的转移到活动分区,并在转移完成后删除固定分区中的数据;

(7)根据数据量进行分裂设计,如果单表数据量太庞大,则把时间跨度大的当前活动分区表分裂成时间跨度小的目标活动分区表,先把目标活动分区表加入继承关系,同时修改定时的数据转移逻辑,数据往目标活动分区表写,然后启动一个事务将当前活动分区表中的数据分裂到目标活动分区表中去,同时断开当前活动分区表的连接;

(8)预先定义好分层和数据量对应规则,当活动分区表数据量超过上限阈值则自动往下一层分裂;

(9)根据分区继承表的异步主动活动的数据转移逻辑来创建异步继承表;

(10)创建用于记录每次活动分区的方法、分区表名和时间戳的元表;

(11)创建用于数据转移的逻辑函数,所述逻辑函数首先从元表获取当前异步分区模式,然后判断是否到达临界点,如果到达临界点则从统计表取上一个分区表的大小,则根据数据量选择对应分区策略创建分区,并在元表新增一条记录来标示当前的分区策略,如果未到达临界点,则根据当前分区策略调用对应的数据转移逻辑,并根据当前时间选择合适的固定分区,在选定的固定分区中取出最小时间和最大时间,并从元表取出最小时间和最大时间范围的分区表名以及前后分区表名,并将固定分区的数据写入这些活动分区表,并同时清除固定分区表的数据;

(12)定时调用数据转移函数。

2.根据权利要求1所述的PostgreSQL大数据高效免维护自动分区方法,其特征在于,使用bulkload的方式数据转移,使用TRUNCATE的方式删除固定分区的数据。

3.根据权利要求1或2或所述的PostgreSQL大数据高效免维护自动分区方法,其特征在于,分区值的固定循环值包括按月循环、按周循环、按日循环、按小时循环、按分钟循环、按秒循环或按半小时循环。

4.根据权利要求3所述的PostgreSQL大数据高效免维护自动分区方法,其特征在于,分区逻辑如果按小时的粒度,只需要创建24个分区。

PostgreSQL大数据高效免维护自动分区方法

技术领域

[0001] 本发明涉及数据库大数据处理技术领域，尤其涉及一种PostgreSQL大数据高效免维护自动分区方法。

背景技术

[0002] 现有的PostgreSQL数据库由于对单个表的记录数量是有限制的，同时存储的空间方面也有限制，即便这些都没有限制，MDL(数据操作语言命令)的操作效率也会随着表的大小增长受到影响。现有PostgreSQL数据库对大数据的处理一般采用数据分区或数据分层的方式来实现。例如，PostgreSQL数据库不仅支持range、list和hash这几种数据分区方式，同时还支持任意组合的数据分层方式，但数据分层方式需要通过触发器来实现。

[0003] 目前，在使用PostgreSQL数据库的传统数据分区时存在以下问题：

[0004] 一是现有PostgreSQL数据库的数据分区方法比较死板，数据库分区表的定义一旦固定下来，后期随着数据量的变化，要分区表的分裂就比较麻烦，需要人为干预，并且会堵塞DML的操作，影响业务。例如，现有PostgreSQL数据库分区表的分区定义和分区约束随着时间的推移，分区表需要人为干预添加，如果要扩展分区，例如从月扩展到日分区需要人为干预，易于出现问题。

[0005] 二是对于范围分区，需要定期增加分区，否则数据会因为值不能落在已定义的分区内而溢出，返回错误，影响业务。即便有默认分区，也影响DML的效率。例如，现有PostgreSQL数据库的分区定义和入口对象绑定，维护起来比较麻烦。

[0006] 三是分区越多，分区逻辑代码就越长，一方面影响效率，另一方面增加了代码维护难度以及出错概率。例如，现有PostgreSQL数据库的分区逻辑随着时间的推移，逻辑代码需要人为干预修改，分区越多，代码越庞大，效率越低，出错概率越大。扩展分区需要人为干预修改分区逻辑。

发明内容

[0007] 本发明是为了解决PostgreSQL数据库的分区表定义固定下来后导致后期分区表不易分裂，数据易从范围分区表中溢出，分区表越多使得分区逻辑的代码就越长从而导致DML的效率低和增加代码维护难度的不足，提供一种将分区逻辑固定到一个循环内，根据固定循环创建固定的分区逻辑，活动分区从分区逻辑中剥离，数据向活动分区自动转移，根据数据量自动分裂分区的PostgreSQL大数据高效免维护自动分区方法。

[0008] 为了实现上述目的，本发明采用以下技术方案：

[0009] PostgreSQL大数据高效免维护自动分区方法，包括如下步骤：

[0010] (1)创建入口表，且应用程序只与入口表交互；

[0011] (2)按照数据库分区表的分区值特征，将分区值抽象转换为一个随时间推移的固定循环值；

[0012] (3)根据固定的循环值创建固定的分区逻辑；

[0013] (4)创建根据入口表的分区逻辑代码和字段值分发的分区继承表；

[0014] (5)把活动分区从分区逻辑中剥离，让活动分区随着时间的推移进行自动创建；

[0015] (6)。利用继承特性将固定分区的数据定时的转移到活动分区。并在转移完成后删除固定分区中的数据；

[0016] (7)根据数据量进行分裂设计，如果单表数据量太庞大，则把时间跨度大的当前活动分区表分裂成时间跨度小的目标活动分区表，先把目标活动分区表加入继承关系，同时修改定时的数据转移逻辑，数据往目标活动分区表写，然后启动一个事务将当前活动分区表中的数据分裂到目标活动分区表中去，同时断开当前活动分区表的连接；

[0017] (8)预先定义好分层和数据量对应规则，当活动分区表数据量超过上限阈值则自动往下一层分裂；

[0018] (9)根据分区继承表的异步主动活动的数据转移逻辑来创建异步继承表；

[0019] (10)创建用于记录每次活动分区的方法、分区表名和时间戳的元表；

[0020] (11)创建用于数据转移的逻辑函数，所述逻辑函数首先从元表获取当前异步分区模式，然后判断是否到达临界点，如果到达临界点则从统计表取上一个分区表的大小，则根据数据量选择对应分区策略创建分区，并在元表新增一条记录来标示当前的分区策略，如果未到达临界点，则根据当前分区策略调用对应的数据转移逻辑，并根据当前时间选择合适的固定分区，在选定的固定分区中取出最小时间和最大时间，并从元表取出最小时间和最大时间范围的分区表名以及前后分区表名，并将固定分区的数据写入这些活动分区表，并同时清除固定分区表的数据；

[0021] (12)定时调用数据转移函数。

[0022] 作为优选，使用bulkload的方式数据转移，使用TRUNCATE的方式删除固定分区的数据。

[0023] 作为优选，分区值的固定循环值包括按月循环、按周循环、按日循环、按小时循环、按分钟循环、按秒循环或按半小时循环。

[0024] 作为优选，分区逻辑如果按小时的粒度，只需要创建24个分区。

[0025] 本发明能够达到如下效果：

[0026] 1、将分区逻辑固定到一个循环内，实现分区逻辑代码免维护。同时缩小了循环区域的数据量，这样的话可以和活动分区区别存储，例如高IO能力的存储和低IO能力的存储。节约高IO能力的存储成本。

[0027] 2、根据固定循环创建固定的分区逻辑，例如按小时的粒度，只需要创建24个分区。分区表和分区逻辑固定后，分区和代码免维护。因为逻辑代码部分相比传统的逻辑更加简洁，还解决了代码效率问题。

[0028] 3、活动分区从分区逻辑中剥离，随着时间的推移自动创建活动分区，解决了分区维护的问题。

[0029] 4、数据向活动分区自动转移，利用继承关系和DDL事务解决数据一致性问题和自动分区的问题。

[0030] 5、根据数据量自动分裂分区，数据量的获取可以从数据库采用数据中得到，分裂阈值可以存储在元表或者逻辑代码中，自动分裂解决了总入口查询效率问题，因为表过多或者单表过大都会带来解析或查询效率问题。

附图说明

[0031] 图1是本发明的一种架构原理示意图。

[0032] 图2是本发明逻辑函数的一种流程原理示意图。

具体实施方式

[0033] 下面通过实施例，并结合附图，对本发明的技术方案作进一步具体的说明。

[0034] 实例一：PostgreSQL大数据高效免维护自动分区方法，参见图1、图2所示，包括如下步骤：

[0035] (1)创建入口表，且业务方并行数据DML的应用程序只与入口表交互。

[0036] (2)按照分区值的特征，将值抽象转换为一个随时间推移的循环值，例如时间字段，可以演化成按时间推移的循环值：按月循环(1-12)，按周循环(1-53)，按日循环(1-366)，按小时循环(1-24)，按分钟循环(1-60)，按秒循环(1-60)，按半小时循环(0100-2400)等，为解决分区逻辑代码维护提供了基础。

[0037] (3)根据固定循环值创建固定的分区逻辑，分区逻辑的分区逻辑代码根据字段值分发到对应的继承表。例如按小时的粒度，只需要创建24个分区。分区表和分区逻辑固定后，分区和代码免维护。解决了效率，分区逻辑代码维护问题。

[0038] (4)创建根据入口表的分区逻辑代码和字段值分发的分区继承表。例如，分发到继承表1和继承表n。

[0039] (5)把活动分区从分区逻辑中剥离，让活动分区随着时间的推移进行自动创建。解决了分区维护的问题。

[0040] (6)利用继承特性将固定分区的数据定时的转移到活动分区，并在转移完成后删除固定分区中的数据。数据向活动分区自动转移的设计，利用继承特性将固定分区的数据定时的转移到活动分区，并从固定分区删除，数据转移可以使用bulkload的方式，加快转移速度，删除固定分区的数据可以选择TRUNCATE方式，加快速度，同时不会阻断DML，解决了自动分区的问题。

[0041] (7)根据数据量进行分裂设计，如果单表数据量太庞大，则把时间跨度大的当前活动分区表分裂成时间跨度小的目标活动分区表，先把目标活动分区表加入继承关系，同时修改定时的数据转移逻辑，数据往目标活动分区表写，然后启动一个事务将当前活动分区表中的数据分裂到目标活动分区表中去，同时断开当前活动分区表的连接。例如当前活动分区是按月份分区的，单表数据量太庞大，要分裂成按日分区，先把日表加入继承关系，同时修改定时数据转移逻辑，数据往日表写，然后启动一个事务将月表数据分裂到日表同时断开月表关系，保证数据一致性，这些操作对应用来说都是透明的，并且不阻断DML，解决传统分区分裂可能阻断DML的问题。

[0042] (8)预先定义好分层和数据量对应规则，当活动分区表数据量超过上限阈值则自动往下一层分裂。例如，预先定义好分层和数据量对应规则，例如年，月，周，日，小时，分钟对应的数据量级的规则，当分区数据量超过上限阈值则往下一层分裂，解决实现自动分裂的问。

[0043] (9)根据分区继承表的异步主动活动的数据转移逻辑来创建异步继承表a1和异步

继承表an。

[0044] (10)创建用于记录每次活动分区的方法、分区表名和时间戳的元表。

[0045] (11)创建用于数据转移的逻辑函数，所述逻辑函数首先从元表获取当前异步分区模式，然后判断是否到达临界点，如果到达临界点则从统计表取上一个分区表的大小，则根据数据量选择对应分区策略创建分区，并在元表新增一条记录来标示当前的分区策略，如果未到达临界点，则根据当前分区策略调用对应的数据转移逻辑，并根据当前时间选择合适的固定分区，在选定的固定分区中取出最小时间和最大时间，并从元表取出最小时间和最大时间范围的分区表名以及前后分区表名，并将固定分区的数据写入这些活动分区表，并同时清除固定分区表的数据。

[0046] (12)定时调用数据转移函数。

[0047] 实例二：PostgreSQL大数据高效免维护自动分区方法，参见图1、图2所示，包括如下步骤：

[0048] 一、与业务交互的入口表和分区逻辑实施。

[0049] 1-1、创建入口表tbl，指定一个分区字段例如ctime，要求可以从字段内容区分该行插入哪个继承表，例如类型为时间戳。应用程序只与入口表交互。

[0050] 1-2、创建入口表的继承表，例如按小时创建24个继承表，tbl_01到tbl_24，为每个继承表的ctime加上必要的约束，例如tbl_01约束checkto_char(ctime,'hh24')＝01。

[0051] 1-3、创建插入触发器函数，根据ctime函数字段值，插入对应的继承表。例如2014-01-0112:01:01插入tbl_12表。

[0052] 1-4、为了防止删除和更新入口表，创建删除，更新触发器函数，当入口表执行删除或更新操作时，直接返回异常。

[0053] 1-5、创建行触发器函数，行触发器函数触发前面创建的插入触发器函数。

[0054] 这个部分创建完后就不需要再维护了，数据插入入口表时，自动插入到对应的继承表。

[0055] 二、异步数据转移逻辑实施。

[0056] 2-1、创建元表，记录异步继承表的数据分区维度(例如日，月，年)，分区表名，时间戳，初始状态。

[0057] 2-2、初始化异步继承表维度，设置为最低的维度(例如日)。

[0058] 2-3、创建用于数据转移逻辑的函数。

[0059] 2-4、函数处理流程：开启ssi或rr事务，从元表获取当前异步分区模式(如日，月，年)，并判断是否到达临界点，如果达到临界点，则从统计表取上一个分区表的大小，然后根据数据量选择对应分区策略，创建分区，并在元素表新增一条记录，当前分区策略。然后根据分区策略，调用对应的数据转移逻辑，然后根据当前时间选择合适的固定分区，在选定的固定分区中取出min和maxtime。然后从元素表取出这个时间段内的分区以及前后增加各一个分区表，以这些分区表和分区策略为基础，将固定分区的数据插入这些分区表，并清除选定固定分区的记录。如果没有达到临界点，则直接进入到根据分区策略，调用对应的数据转移逻辑这一步。在本实例中，函数处理流程先从元表获取当前异步继承表分区维度，初始维度为日。若当前为非初始维度：根据当前的异步继承表的分区维度，从统计表获取上一个异步继承表分区的容量大小，若当前为初始维度：初始维度创建发生3天后，取前一天的异步

继承表分区的容量大小。对照阈值判断是否需要变更维度，例如日表容量小于阈值，变更为月表；月表容量大于阈值，变更为日表，容量小于阈值变更为年表；年表容量大于阈值变更为月表。如果异步继承表的维度发生了变更，将变更写入元表，作为当前的异步继承表的维度。从继承表前2个分区数据中取出min，max时间，例如现在是17点，取15的分区数据。判断继承表数据是否跨异步继承表分区当前维度，(例如min，max包含了2天的数据，则跨日维度)。如果跨维度，判断维度对应的异步继承表是否全部存在(例如tbl_20140101，tbl_20140102)，不存在则创建对应的异步继承表，并添加相应的约束。数据插入对应的异步继承表。truncate该转移数据的继承表分区，截断数据。

[0060] 2-5开启repeatableread或ssi事务，调用数据转移函数。

[0061] 2-6将步骤2-4做成定时任务，定时执行(但是不能超越1天)，建议1个小时执行一次。

[0062] 实例一是将分区逻辑固定到一个循环内，实现分区逻辑代码免维护。同时缩小了循环区域的数据量，这样的话可以和活动分区区别存储，例如高IO能力的存储和低IO能力的存储。节约高IO能力的存储成本。二是根据固定循环创建固定的分区逻辑，例如按小时的粒度，只需要创建24个分区。分区表和分区逻辑固定后，分区和代码免维护。因为逻辑代码部分相比传统的逻辑更加简洁，还解决了代码效率问题。三是把活动分区从分区逻辑中剥离，随着时间的推移自动创建活动分区，解决了分区维护的问题。四是把数据向活动分区自动转移，利用继承关系和DDL事务解决数据一致性问题和自动分区的问题。五是根据数据量自动分裂分区，数据量的获取可以从数据库采用数据中得到，分裂阈值可以存储在元表或者逻辑代码中，自动分裂解决了总入口查询效率问题，因为表过多或者单表过大都会带来解析或查询效率问题。

[0063] 上面结合附图描述了本发明的实施方式，但实现时不受上述实施例限制，本领域普通技术人员可以在所附权利要求的范围内做出各种变化或修改。

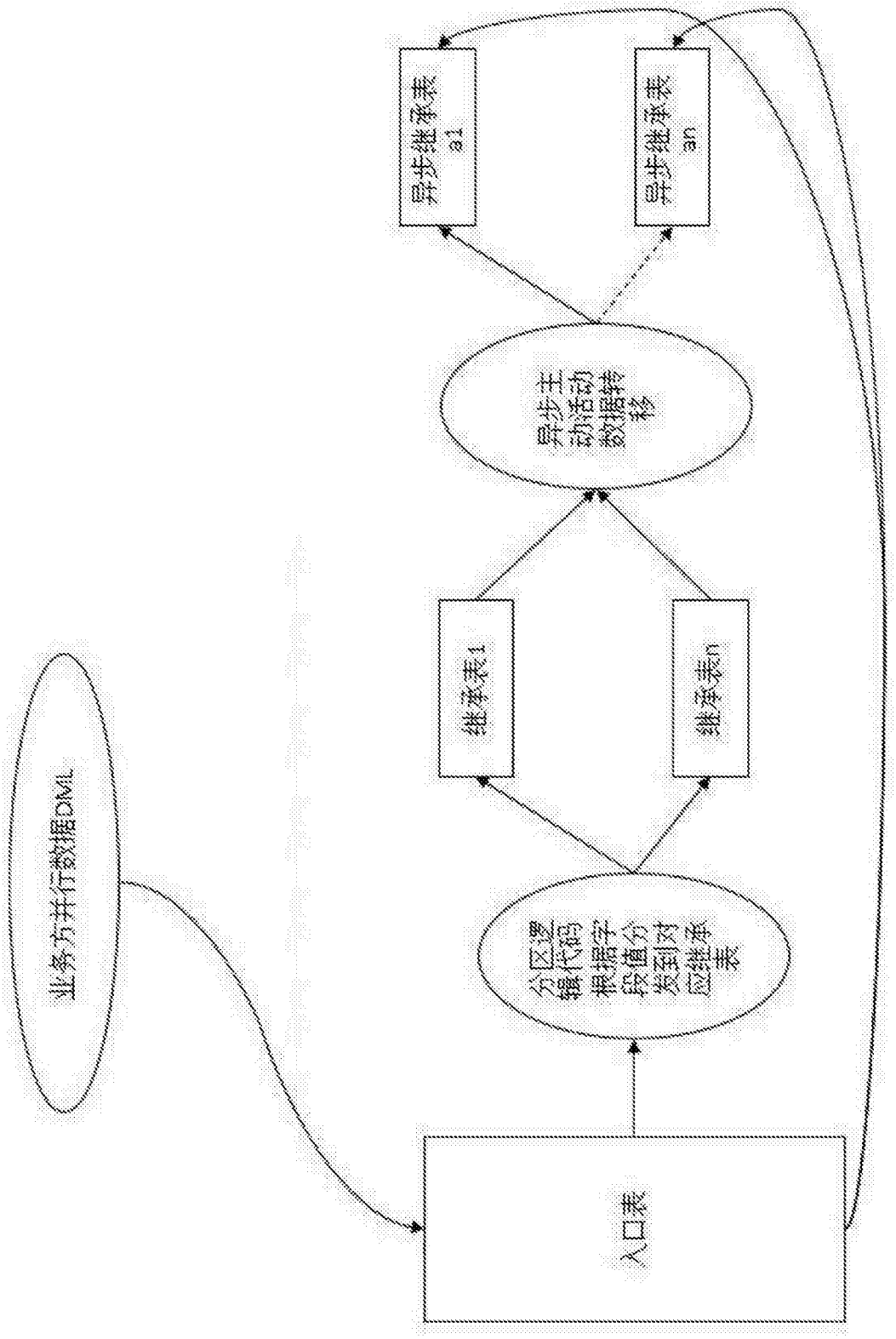

异步继承表 a1
异步继承表 an
异步主动活动数据转移
继承表1
继承表n
分区逻辑代码根据字符值分发到对应继承表
业务方并行数据DML
入口表

图1

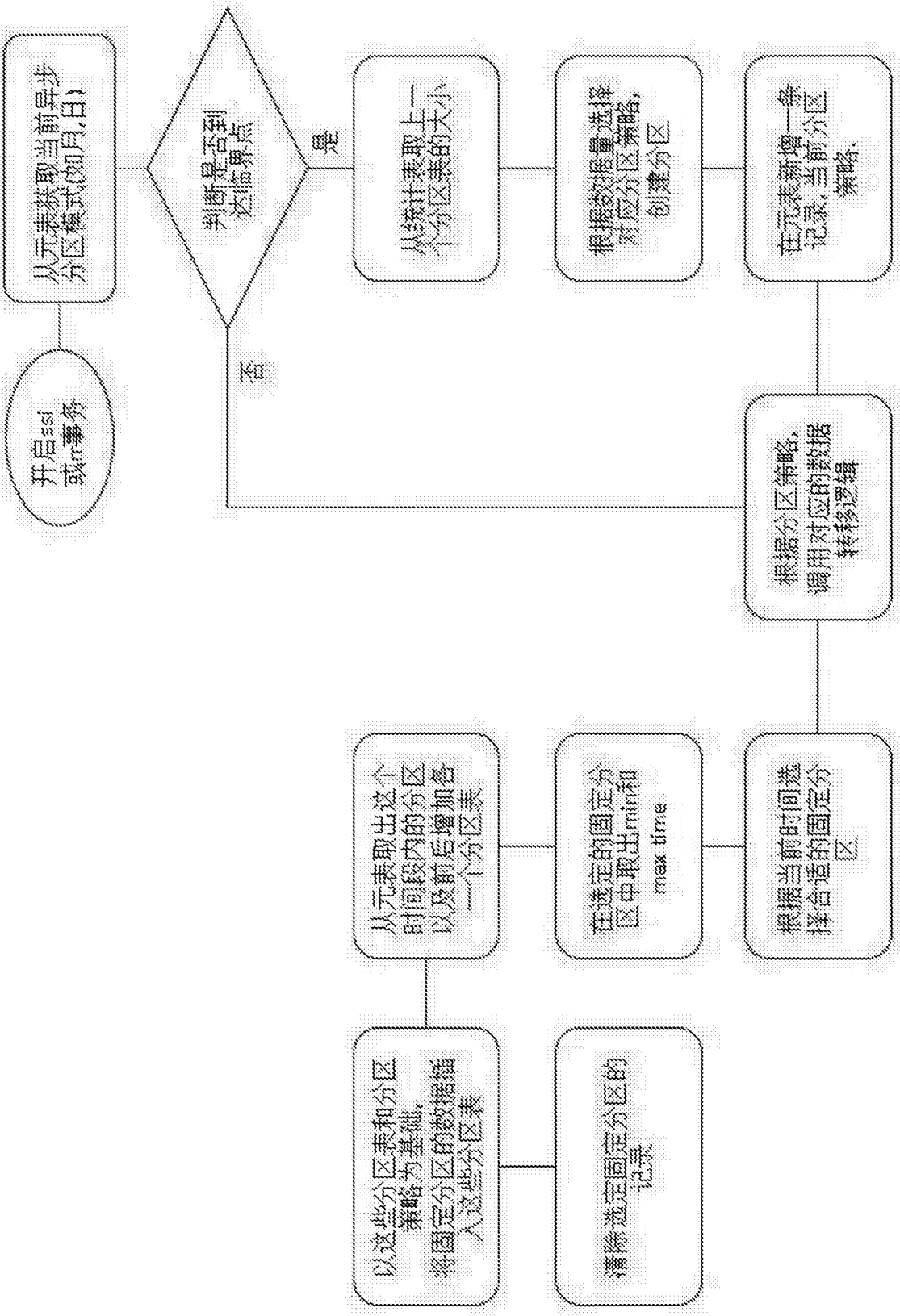

从元表获取当前异步分区模式(如月,日)
开启esl或rr事务
判断是否到达临界点
是
否
从统计表取上一个分区表的大小
根据数据量选择对应分区策略，创建分区
在元表新增一条记录，当前分区策略，
根据分区策略，调用对应的数据转移逻辑
根据当前时间选择合适的固定分区
在选定的固定分区中取出min和max time
从元表取出这个时间段内的分区以及前后增加各一个分区表
以这些分区表和分区策略为基础，将固定分区的数据插入这些分区表
清除选定固定分区的记录

图2